

Unit-6: SQL

Data Definition and Data Types:

SQL (Structured Query Language) is a language used to interact with relational databases. SQL uses the terms **table**, **row**, and **column** for the formal relational model terms **relation**, **tuple**, and **attribute**, respectively. We will use the corresponding terms inter-changeably. The main SQL command for data definition is the CREATE statement, which can be used to create schemas, tables (relations), and domains (as well as other constructs such as views, assertions, and triggers).

SQL Data Types:

Data types are used to represent the nature of the data that can be stored in the database table. For example, in a particular column of a table, if we want to store a string type of data then we will have to declare a string data type of this column.

Here are the most common SQL data types:

Data Type	Properties
Numeric data types	These are used to store numeric values. Examples include INT , BIGINT , DECIMAL , and FLOAT .
Character data types	These are used to store character strings. Examples include CHAR , VARCHAR , and TEXT .
Date and time data types	These are used to store date and time values. Examples include DATE , TIME , and TIMESTAMP .
Binary data types	These are used to store binary data, such as images or audio files. Examples include BLOB and BYTEA .

Data Type	Properties
Boolean data type	This data type is used to store logical values. The only possible values are <i>TRUE</i> and <i>FALSE</i> .
Interval data types	These are used to store intervals of time. Examples include <i>INTERVAL YEAR</i> , <i>INTERVAL MONTH</i> , and <i>INTERVAL DAY</i> .
Array data types	These are used to store arrays of values. Examples include <i>ARRAY</i> and <i>JSON</i> .
XML data type	This data type is used to store <i>XML</i> data.
Spatial data types	These are used to store geometric or geographic data. Examples include <i>POINT</i> , <i>LINE</i> , and <i>POLYGON</i> .

Different databases may have different variations of these data types, or they may have additional data types not listed here. Understanding SQL data types are important for creating tables and working with data in a database, as it affects how data is stored and processed.

Like in other programming languages, SQL also has certain datatypes available. A brief idea of all the datatypes is discussed below.

Binary Datatypes:

There are four subtypes of this datatype which are given below:

Data Type	Description
Binary	The maximum length of 8000 bytes(Fixed-Length binary data)
varbinary	The maximum length of 8000 bytes(Variable Length binary data)
varbinary(max)	The maximum length of 231 bytes(SQL Server 2005 only).(Variable Length binary data)
image	Maximum Length of 2,147,483,647 bytes(Variable Length binary data)

Exact Numeric Datatype:

There are nine subtypes which are given below in the table. The table contains the range of data in a particular type.

Data Type	From	To
BigInt	-2^{63} (-9,223,372,036,854,775,808)	2^{62} (9,223,372,036,854,775,807)
Int	-2^{31} (-2,147,483,648)	2^{30} (2,147,483,647)

Data Type	From	To
smallint	-2 ¹⁵ (-32,768)	2 ¹⁴ (32,767)
tinyint	0	2 ⁷
bit	0	1
money	-922,337,203,685,477,5808	922,337,203,685,477,5808
smallmoney	-2,147,748	2,147,748

Character String Datatype:

The subtypes are given in below table

Data Type	Description
char	The maximum length of 8000 characters.(Fixed-Length non-Unicode Characters)
varchar	The maximum length of 8000 characters.(Variable-Length non-Unicode Characters)
varchar(max)	The maximum length of 231 characters(SQL Server 2005 only).(Variable Length non-Unicode data)

text	The maximum length of 2,127,483,647 characters(Variable Length non-Unicode data)
------	--

Unicode Character String Datatype:

The details are given in below table

Data Type	Description
Nchar	The maximum length of 4000 characters(Fixed-Length Unicode Characters)
Nvarchar	The maximum length of 4000 characters.(Variable-Length Unicode Characters)
nvarchar(max)	The maximum length of 231 characters(SQL Server 2005 only).(Variable Length Unicode data)

Date and Time Datatype:

The details are given in the below table.

Data Type	Description
DATE	A data type is used to store the data of date in a record
TIME	A data type is used to store the data of time in a record
DATETIME	A data type is used to store both the data, date, and time in the record.

XML Datatype

XML data type allows storage of XML documents and fragments in a SQL Server database

DataType	Description
XML Datatype	A Datatype used to store data in the format of XML datatype

Spatial Datatype:

A datatype is used for storing planar spatial data, such as points, lines, and polygons, in a database table.

DataType	Description
Geometry	A datatype is used for storing planar spatial data, such as points, lines, and polygons, in a database table.

Array Datatype:

SQL Server does not have a built-in array datatype. However, it is possible to simulate arrays using tables or XML data types.

Specifying Constraints:

In a database table, we can add rules to a column known as constraints. These rules control the data that can be stored in a column. We can specify the limit on the type of data that can be stored in a particular column in a table using constraints.

The available constraints in SQL are:

- **NOT NULL:** This constraint tells that we cannot store a null value in a column. That is, if a column is specified as NOT NULL then we will not be able to store null in this particular column any more.
- **UNIQUE:** This constraint when specified with a column, tells that all the values in the column must be unique. That is, the values in any row of a column must not be repeated.
- **PRIMARY KEY:** A primary key is a field which can uniquely identify each row in a table. And this constraint is used to specify a field in a table as primary key.
- **FOREIGN KEY:** A Foreign key is a field which can uniquely identify each row in the another table. And this constraint is used to specify a field as Foreign key.
- **CHECK:** This constraint helps to validate the values of a column to meet a particular condition. That is, it helps to ensure that the value stored in a column meets a specific condition.
- **DEFAULT:** This constraint specifies a default value for the column when no value is specified by the user.

Syntax:

Below is the syntax to create constraints using CREATE TABLE statement at the time of creating the table.

```
CREATE TABLE sample_table  
(  
    column1 data_type(size) constraint_name,  
    column2 data_type(size) constraint_name,  
    column3 data_type(size) constraint_name,  
    ....  
);
```

Where.

sample_table: Name of the table to be created.

data_type: Type of data that can be stored in the field.

constraint_name: Name of the constraint. for example- NOT NULL, UNIQUE, PRIMARY KEY etc.

Example of each constraints:

NOT NULL Constraint:

If we specify a field in a table to be NOT NULL, then the field will never accept null value. That is, you will be not allowed to insert a new row in the table without specifying any value to this field.

For example, the below query creates a table Student with the fields ID and NAME as NOT NULL. That is, we are bound to specify values for these two fields every time we wish to insert a new row.

```
CREATE TABLE Student  
(  
    ID int(6) NOT NULL,  
    NAME varchar(10) NOT NULL,  
    ADDRESS varchar(20)  
);
```

UNIQUE Constraint:

This constraint helps to uniquely identify each row in the table. i.e. for a particular column, all the rows should have unique values. We can have more than one UNIQUE columns in a table.

For example, the below query creates a table Student where the field ID is specified as UNIQUE. i.e, no two students can have the same ID.

```
CREATE TABLE Student  
(  
    ID int(6) NOT NULL UNIQUE,  
    NAME varchar(10),  
    ADDRESS varchar(20)  
);
```

PRIMARY KEY Constraint:

Primary Key is a field which uniquely identifies each row in the table. If a field in a table as primary key, then the field will not be able to contain NULL values as well as all the rows should have unique values for this field. So, in other words we can say that this is combination of NOT NULL and UNIQUE constraints. A table can have only one field as primary key. Below query will create a table named Student and specifies the field ID as primary key.

```
CREATE TABLE Student  
(  
    ID int(6) NOT NULL UNIQUE,  
    NAME varchar(10),  
    ADDRESS varchar(20),  
    PRIMARY KEY(ID)  
);
```

FOREIGN KEY Constraint:

Foreign Key is a field in a table which uniquely identifies each row of a another table. That is, this field points to primary key of another table. This usually creates a kind of link between the tables.

Consider the two tables as shown below:

Orders:

O_ID	ORDER_NO	C_ID
1	2253	3
2	3325	3
3	4521	2
4	8532	1

Customers:

C_ID	NAME	ADDRESS
1	RAMESH	KATHMANDU
2	SURESH	POKHARA
3	DHARMESH	BUTWAL

As we can see clearly that the field C_ID in Orders table is the primary key in Customers table, i.e. it uniquely identifies each row in the Customers table. Therefore, it is a Foreign Key in Orders table.

```
CREATE TABLE Orders
```

```
(
```

```
    Order_ID int NOT NULL,
```

```
    Order_No int NOT NULL,
```

```
    Customer_ID int,
```

```
    PRIMARY KEY (Order_ID),
```

```
    FOREIGN KEY (Customer_ID) REFERENCES Customers (Customer_ID)
```

```
);
```

CHECK Constraint:

Using the CHECK constraint we can specify a condition for a field, which should be satisfied at the time of entering values for this field.

For example, the below query creates a table Student and specifies the condition for the field AGE as (AGE >= 18). That is, the user will not be allowed to enter any record in the table with AGE < 18.

```
CREATE TABLE Student
(
    ID int(6) NOT NULL,
    NAME varchar(10) NOT NULL,
    AGE int NOT NULL CHECK (AGE >= 18)
);
```

DEFAULT Constraint:

This constraint is used to provide a default value for the fields. That is, if at the time of entering new records in the table if the user does not specify any value for these fields then the default value will be assigned to them.

For example, the below query will create a table named Student and specify the default value for the field AGE as 18.

```
CREATE TABLE Student
(
    ID int(6) NOT NULL,
    NAME varchar(10) NOT NULL,
    AGE int DEFAULT 18
);
```

Alias in the Sql Query:

We can use short name of the table called alias.

Syntax: table_name alias_name

For example:

```
select* from country c inner join state s on c.c_id=s.c_id;
```

Here, c and s are the alias of country and state table respectively.

	c_name	s_name
1	Nepal	Bagmati
2	Nepal	Lumbini
3	Nepal	Kamali
4	India	Maharastra
5	India	Kamatak
6	China	Guangdong

Custom heading of the resultant table:

We can make the custom heading of the table to make table heading more readable.

Syntax: column_name as custom_col_name

For example:

```
select c.c_name as Country, s.s_name as State from country c inner join state s
on c.c_id=s.c_id;
```

Here, Country and States are Custom column name

	Country	State
1	Nepal	Bagmati
2	Nepal	Lumbini
3	Nepal	Kamali
4	India	Maharastra
5	India	Kamatak
6	China	Guangdong

Basic Retrieval Queries in SQL:***Select Statement:***

The SELECT statement is used to select data from a database or retrieve data from database. The data retrieved is stored in a result table, called the result-set.

Syntax:

SELECT col1, col2, col3 FROM table_name;

Example 1: Select all data from table

Query: select* from student;

Output:

Results		Messages				
	s_id	sname	roll_no	saddress	sphone	sdepartment
1	1	Rahul Karki	1	ktm	9851223567	BCA
2	2	Sudan Rana	2	bhaktapur	9858223567	BCA
3	3	Sankar Shrestha	3	ktm	9851223667	BCA
4	4	Gopal Chand	1	Rolpa	9851253567	CSIT
5	10	Rasila Karki	1	ktm	9851228567	BBA
6	11	Sunita Rana	2	bhaktapur	9858213567	CSIT
7	12	Shristi Shrestha	3	ktm	9851283667	BBA
8	13	Birat Shah	5	Rolpa	9851250567	CSIT

Example 2: Select specific column from table

Query: select sname, roll_no from student;

Results		Messages	
	sname	roll_no	
1	Rahul Karki	1	
2	Sudan Rana	2	
3	Sankar Shrestha	3	
4	Gopal Chand	1	
5	Rasila Karki	1	
6	Sunita Rana	2	
7	Shristi Shrestha	3	
8	Birat Shah	5	

Example 3: Make customize result heading**Query:**

```
select sname as Name, roll_no as Roll_No, saddress as Address from student;
```

	Name	Roll_No	Address
1	Rahul Karki	1	ktm
2	Sudan Rana	2	bhaktapur
3	Sankar Shrestha	3	ktm
4	Gopal Chand	1	Rolpa
5	Rasila Karki	1	ktm
6	Sunita Rana	2	bhaktapur
7	Shristi Shrestha	3	ktm
8	Birat Shah	5	Rolpa

Example 4: Display the name and department of student who are from CSIT department**Query:**

```
select sname as Name, sdepartment as Department from student where sdepartment = 'CSIT';
```

	Name	Department
1	Gopal Chand	CSIT
2	Sunita Rana	CSIT
3	Birat Shah	CSIT

Example 5: Display the name and roll no of student whose roll num is less than 3

Query: `select sname as Name, roll_no as Roll_No from student where roll_no<3;`

	Name	Roll_No
1	Rahul Karki	1
2	Sudan Rana	2
3	Gopal Chand	1
4	Rasila Karki	1
5	Sunita Rana	2

Example 6: Display the information of student who is from ktm and study in BCA

Query: `select* from student where saddress = 'ktm' and sdepartment='BCA';`

Results		Messages				
	s_id	sname	roll_no	saddress	sphone	sdepartment
1	1	Rahul Karki	1	ktm	9851223567	BCA
2	3	Sankar Shrestha	3	ktm	9851223667	BCA

Example 7: Display the information of student either who is form ktm or study in CSIT

Query: `select* from student where saddress = 'ktm' or sdepartment='CSIT';`

Results		Messages				
	s_id	sname	roll_no	saddress	sphone	sdepartment
1	1	Rahul Karki	1	ktm	9851223567	BCA
2	3	Sankar Shrestha	3	ktm	9851223667	BCA
3	4	Gopal Chand	1	Rolpa	9851253567	CSIT
4	10	Rasila Karki	1	ktm	9851228567	BBA
5	11	Sunita Rana	2	bhaktapur	9858213567	CSIT
6	12	Shristi Shrestha	3	ktm	9851283667	BBA
7	13	Birat Shah	5	Rolpa	9851250567	CSIT

Example 8: Display the information of student who is from out of ktm

Query: `select* from student where saddress!='ktm';`

Results		Messages				
	s_id	sname	roll_no	saddress	sphone	sdepartment
1	2	Sudan Rana	2	bhaktapur	9858223567	BCA
2	4	Gopal Chand	1	Rolpa	9851253567	CSIT
3	11	Sunita Rana	2	bhaktapur	9858213567	CSIT
4	13	Birat Shah	5	Rolpa	9851250567	CSIT

Complex Retrieval Queries:

Consider we have following table

Results	Messages
dept_id	dept_name
1	HR
2	Account
3	Logistics
4	Supply
5	Managerial

Results	Messages
desig_id	desig_name
1	CEO
2	BOD
3	HOD
4	Project Le...
5	Ast. Leader
6	Senior Emp
7	Junior Emp
8	Intern

Results	Messages
emptype_id	emptype_name
1	Full Time
2	Part Time
3	Volunteer
4	Part Time
5	Full Time
6	Part Time
7	HR

Department

Designation

Employee Type

Results

Messages

	proj_id	proj_name	start_date	deadline
1	1	Library Management System	2022-01-01	2022-06-30
2	2	Hospital Management System	2022-07-25	2023-04-26
3	3	Paisa Digital wallet	2023-01-12	2023-09-30

Project

Results

Messages

	emp_id	emp_name	emp_address	emp_contact	emp_salary	dept_id	desig_id	emptype_id	proj_id
1	1	Rahul	kathmandu	9845098976	50000	1	6	1	1
2	2	Gopal	bhaktapur	9845048976	60000	2	5	1	1
3	3	Kiran	dolakha	9845598976	70000	4	6	1	2
4	4	Prakriti	kathmandu	9845798976	40000	1	8	1	3
5	5	Samjhana	dang	9845898976	90000	5	7	2	1
6	6	Sudhir	surkhet	9845298976	30000	2	8	1	3
7	7	Sampanna	bhaktapur	9845398976	50000	1	6	2	2
8	8	Zakir	butwal	984578976	50000	3	6	2	1

Employee

Example 1: Display the detail of the employee whose salary is less than 50k

Query:

```
select
employee.emp_name as Name,
employee.emp_address as Address,
employee.emp_contact as Contact,
employee.emp_salary as Salary,
department.dept_name as Department,
designation.desig_name as Designation,
employee_type.emptype_name as Type,
project.proj_name as Project
from employee, department, designation, employee_type, project
where employee.dept_id=department.dept_id
and employee.desig_id=designation.desig_id
and employee.emptype_id=employee_type.emptype_id
and employee.proj_id=project.proj_id
and employee.emp_salary<50000;
```

	Name	Address	Contact	Salary	Department	Designation	Type	Project
1	Prakriti	kathmandu	9845798976	40000	HR	Intern	Full Time	Paisa Digital wallet
2	Sudhir	surkhet	9845298976	30000	Account	Intern	Full Time	Paisa Digital wallet

Example 2: Display the detail of the employee whose salary is between 30k and 80k

Query:

```
select
employee.emp_name as Name,
employee.emp_address as Address,
employee.emp_contact as Contact,
employee.emp_salary as Salary,
department.dept_name as Department,
designation.desig_name as Designation,
employee_type.emptype_name as Type,
project.proj_name as Project
from employee, department, designation, employee_type, project
where employee.dept_id=department.dept_id
and employee.desig_id=designation.desig_id
and employee.emptype_id=employee_type.emptype_id
and employee.proj_id=project.proj_id
and employee.emp_salary>30000 and employee.emp_salary<80000;
```

Results		Messages						
	Name	Address	Contact	Salary	Department	Designation	Type	Project
1	Rahul	kathmandu	9845098976	50000	HR	Senior Emp	Full Time	Library Management System
2	Gopal	bhaktapur	9845048976	60000	Account	Ast. Leader	Full Time	Library Management System
3	Kiran	dolakha	9845598976	70000	Supply	Senior Emp	Full Time	Hospital Management System
4	Prakriti	kathmandu	9845798976	40000	HR	Intern	Full Time	Paisa Digital wallet
5	Sampanna	bhaktapur	9845398976	50000	HR	Senior Emp	Part Time	Hospital Management System
6	Zakir	butwal	984578976	50000	Logistics	Senior Emp	Part Time	Library Management System

Example 3: Display the project detail whose deadline is within 2 month

Query:

`select * from project`

`where`

`deadline < DATEADD(DAY, 60, GETDATE()) and deadline > GETDATE();`

	proj_id	proj_name	start_date	deadline
1	4	Library Management System	2022-01-01	2023-06-30
2	5	Hospital Management System	2022-07-25	2023-06-26
3	6	Paisa Digital wallet	2023-01-12	2023-07-30
4	7	Library Management System	2022-01-01	2023-06-30

Join Query:

Consider two tables Country and State

	c_id	c_name
1	1	Nepal
2	2	India
3	3	China
4	4	Bhutan

	s_id	s_name	c_id
1	9	Bagmati	1
2	10	Lumbini	1
3	11	Kamali	1
4	12	Toronto	NULL
5	13	Maharastra	2
6	14	Karnatak	2
7	15	Guangdong	3
8	16	Sydney	NULL

Country

State

Inner Join:

The INNER JOIN keyword selects records that have matching values in both tables. We can write just JOIN which indicates INNER JOIN.

Example 1: Display all the country name and their states name

Query:

```
select* from country c inner join state s on c.c_id=s.c_id;
```

OR,

```
select* from country c join state s on c.c_id=s.c_id;
```

	c_id	c_name	s_id	s_name	c_id
1	1	Nepal	9	Bagmati	1
2	1	Nepal	10	Lumbini	1
3	1	Nepal	11	Kamali	1
4	2	India	13	Maharashtra	2
5	2	India	14	Karnatak	2
6	3	China	15	Guangdong	3

Here we can see the Bhutan from country table is excluded because it does not have any state name. Similarly Toronto and Sydney also excluded from state table because they do not have any country.

This query can write by using just join keyword as

Example 2: Display all the states name of country Nepal

Query:

```
select c.c_name as Country, s.s_name as State from country c join state s
```

```
on c.c_id=s.c_id
```

```
where c.c_id=1;
```

	Country	State
1	Nepal	Bagmati
2	Nepal	Lumbini
3	Nepal	Kamali

Left Join:

Left Join display the data present in both tables and also displays the remaining data of left table.

Left join = Inner join + remaining data of left table

Example: Display all the list of country and their corresponding state names

Query:

```
select c.c_name as Country, s.s_name as State from country c left join state s
on c.c_id=s.c_id;
```

	Country	State
1	Nepal	Bagmati
2	Nepal	Lumbini
3	Nepal	Kamali
4	India	Maharashtra
5	India	Karnatak
6	China	Guangdong
7	Bhutan	NULL

This output indicates that the Country Bhutan with corresponding NULL value in State do not have any State in State Table. This result displays the all the value of left table but only the condition matched value of right table.

Right Join:

Right Join display the data present in both tables and also displays the remaining data of right table.

Right join = Inner join + remaining data of right table

Example: Display all the list of States with the their corresponding Country name

Query:

```
select c.c_name as Country, s.s_name as State from country c right join state s
on c.c_id=s.c_id;
```

	Country	State
1	Nepal	Bagmati
2	Nepal	Lumbini
3	Nepal	Kamali
4	NULL	Toronto
5	India	Maharastra
6	India	Kamatak
7	China	Guangdong
8	NULL	Sydney

This displays the all the value of right table but only the condition matched value of left table.

Full Outer Join:

Full Outer Join displays the data present in both the table and remaining data of both the left and right tables.

Full Outer Join = Inner Join + Remaining of Left table + Remaining of Right table

Query:

```
select c.c_name as Country, s.s_name as State from country c full outer join state s
on c.c_id=s.c_id;
```

	Country	State
2	Nepal	Lumbini
3	Nepal	Kamali
4	India	Maharastra
5	India	Kamatak
6	China	Guangdong
7	Bhutan	NULL
8	NULL	Toronto
9	NULL	Sydney

Insert, Delete and Update Statement:

Insert Statement:

The INSERT INTO statement is used to insert new records in a table.

INSERT INTO Syntax:

It is possible to write the INSERT INTO statement in two ways:

1. Specify both the column names and the values to be inserted this method is used when we insert values in some specific columns.

```
INSERT INTO table_name (column1, column2, column3,... ....)  
VALUES (value1, value2, value3,...);
```

2. If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table. Here, the INSERT INTO syntax would be as follows:

```
INSERT INTO table_name  
VALUES (value1, value2, value3,...);
```

Example 1: Insert data into t_name, t_phone, t_subject column of the teacher table

Query:

```
insert into teacher(t_name,t_phone, t_subject) values  
('Pratik',8976543567,'DBMS'),  
('Teksan',8976546278, 'AI'),  
('Sujesh',980765247,'Dot Net'),  
('Hiranya',980877652,");
```

Query: select* from teacher;

Results		Messages		
	t_id	t_name	t_phone	t_subject
1	1	Pratik	8976543567	DBMS
2	2	Teksan	8976546278	AI
3	3	Sujesh	980765247	Dot Net
4	4	Hiranya	980877652	

Example 2: Insert data into teacher table

Query:

insert into teacher values

('Sunidhi',8973543567,'DBMS'),

('Rolisha',8976646278, 'AI'),

('Dirgha',980775247,'Dot Net'),

('Sudarshan',986877652,'Java');

Here, we do no mention t_id because it is auto increment so, it will take incremental value automatically.

Query: select* from teacher;

	t_id	t_name	t_phone	t_subject
1	1	Pratik	8976543567	DBMS
2	2	Teksan	8976546278	AI
3	3	Sujesh	980765247	Dot Net
4	4	Hiranya	980877652	
5	5	Sunidhi	8973543567	DBMS
6	6	Rolisha	8976646278	AI
7	7	Dirgha	980775247	Dot Net
8	8	Sudarshan	986877652	Java

Update Statement:

The UPDATE statement is used to modify the existing records in a table.

UPDATE Syntax

UPDATE table_name

SET column1 = value1, column2 = value2,...

WHERE condition;

Example 1: Update t_subject whose id is 4

Query: update teacher set t_subject='C Programing' where t_id=4;

Query: select* from teacher;

	t_id	t_name	t_phone	t_subject
1	1	Pratik	8976543567	DBMS
2	2	Teksan	8976546278	AI
3	3	Sujesh	980765247	Dot Net
4	4	Hiranya	980877652	C Programing
5	5	Sunidhi	8973543567	DBMS
6	6	Rolisha	8976646278	AI
7	7	Dirgha	980775247	Dot Net
8	8	Sudarshan	986877652	Java

Example 2: Update multiple columns

Query:

update teacher set t_subject='C Programing', t_phone=1234567895 where t_id=1;

Query: select* from teacher;

Results		Messages		
	t_id	t_name	t_phone	t_subject
1	1	Pratik	1234567895	C Programing
2	2	Teksan	8976546278	AI
3	3	Sujesh	980765247	Dot Net
4	4	Hiranya	980877652	C Programing
5	5	Sunidhi	8973543567	DBMS
6	6	Rolisha	8976646278	AI
7	7	Dirgha	980775247	Dot Net
8	8	Sudarshan	986877652	Java

Delete Statement:

The DELETE statement is used to delete existing records in a table.

DELETE Syntax

DELETE FROM table_name WHERE condition;

Example: Delete data from table teacher table whose id is 5

Query: delete from teacher where t_id=5;

Query: select* from teacher;

Results		Messages		
	t_id	t_name	t_phone	t_subject
1	1	Pratik	1234567895	C Programing
2	2	Teksan	8976546278	AI
3	3	Sujesh	980765247	Dot Net
4	4	Hiranya	980877652	C Programing
5	6	Rolisha	8976646278	AI
6	7	Dirgha	980775247	Dot Net
7	8	Sudarshan	986877652	Java

ORDER BY Clause:

The ORDER BY keyword is used to sort the result-set in ascending or descending order.

The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

ORDER BY Syntax:

```
SELECT column1, column2,...
FROM table_name
ORDER BY column_name ASC|DESC;
```

Note that the column_name after order by clause should be unique or use only one column_name

Example:**Query 1:** `select* from teacher order by t_name;`

Results		Messages		
	t_id	t_name	t_phone	t_subject
1	7	Dirgha	980775247	Dot Net
2	4	Hiranya	980877652	C Programing
3	1	Pratik	1234567895	C Programing
4	6	Rolisha	8976646278	AI
5	8	Sudarshan	986877652	Java
6	3	Sujesh	980765247	Dot Net
7	2	Teksan	8976546278	AI

Query 2: `select* from teacher order by t_name desc;`

Results		Messages		
	t_id	t_name	t_phone	t_subject
1	2	Teksan	8976546278	AI
2	3	Sujesh	980765247	Dot Net
3	8	Sudarshan	986877652	Java
4	6	Rolisha	8976646278	AI
5	1	Pratik	1234567895	C Programing
6	4	Hiranya	980877652	C Programing
7	7	Dirgha	980775247	Dot Net

Query 3: `select* from teacher order by t_subject;`

Results		Messages		
	t_id	t_name	t_phone	t_subject
1	2	Teksan	8976546278	AI
2	6	Rolisha	8976646278	AI
3	1	Pratik	1234567895	C Programing
4	4	Hiranya	980877652	C Programing
5	3	Sujesh	980765247	Dot Net
6	7	Dirgha	980775247	Dot Net
7	8	Sudarshan	986877652	Java

Query 4: `select* from teacher order by t_subject desc;`

Results		Messages		
	t_id	t_name	t_phone	t_subject
1	8	Sudarshan	986877652	Java
2	7	Dirgha	980775247	Dot Net
3	3	Sujesh	980765247	Dot Net
4	4	Hiranya	980877652	C Programing
5	1	Pratik	1234567895	C Programing
6	2	Teksan	8976546278	AI
7	6	Rolisha	8976646278	AI

Group By Clause:

The GROUP BY statement groups the rows that have the same values into summary rows, like "find the number of teachers teach to each subject".

The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns.

GROUP BY Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Example 1: Find the number of teacher teach to each subjects from teacher table

Query:

```
select t_subject as Subject, COUNT(t_name) as No_of_Teacher
from teacher
group by t_subject
order by COUNT(t_name) desc;
```

	Subject	No_of_Teacher
1	DBMS	4
2	Java	4
3	Dot Net	2
4	AI	2
5	C Programing	2
6	PHP	1

Implementation of COUNT(), MAX(), MIN(), SUM(), AVG() functions using Order by and Group by clauses.

Query: select* from employee;

	emp_id	emp_name	emp_address	emp_contact	emp_salary	dept_id	desig_id	emptype_id	proj_id
1	1	Rahul	kathmandu	9845098976	50000	1	6	1	1
2	2	Gopal	bhaktapur	9845048976	60000	2	5	1	1
3	3	Kiran	dolakha	9845598976	70000	4	6	1	2
4	4	Prakriti	kathmandu	9845798976	40000	1	8	1	3
5	5	Samjhana	dang	9845898976	90000	5	7	2	1
6	6	Sudhir	surkhet	9845298976	30000	2	8	1	3
7	7	Sampanna	bhaktapur	9845398976	50000	1	6	2	2
8	8	Zakir	butwal	984578976	50000	3	6	2	1

Example 1: Display the number of employees from different locations

Query:

```
select emp_address as Employee_Address, COUNT(emp_name) as
No_of_Employees
from employee
group by emp_address;
```

	Employee_Address	No_of_Employees
1	bhaktapur	2
2	butwal	1
3	dang	1
4	dolakha	1
5	kathmandu	2
6	surkhet	1

Example 2: Display the employee name and their salary in descending order

Query: `select emp_name, emp_salary from employee order by emp_salary desc;`

	emp_name	emp_salary
1	Samjhana	90000
2	Kiran	70000
3	Gopal	60000
4	Rahul	50000
5	Sampanna	50000
6	Zakir	50000
7	Prakriti	40000
8	Sudhir	30000

Example 3: Display the name of employee who have maximum salary

Query:

`select emp_name as Employee_Name, emp_salary as Highest_Salary`

`from employee`

`where emp_salary= (select max (emp_salary) from employee);`

	Employee_Name	Highest_Salary
1	Samjhana	90000

Example 4: Display the name of employee who have minimum salary

Query:

```
select emp_name as Employee_Name, emp_salary as Lowest_Salary
from employee
where emp_salary= (select min (emp_salary) from employee);
```

Results		Messages
	Employee_Name	Lowest_Salary
1	Sudhir	30000

Example 5: Find the total salary of the employees

Query: select SUM(emp_salary) as Total_Salary_of_Employees from employee;

	Total_Salary_of_Employees
1	440000

Example 6: Find the average salary of the employees

Query: select AVG(emp_salary) as Average_Salary from employee;

Results		Messages
	Average_Salary	
1	55000	

Views:

In SQL, a view is a virtual table based on the result-set of an SQL statement.

A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

You can add SQL statements and functions to a view and present the data as if the data were coming from one single table.

A view is created with the CREATE VIEW statement.

Note: A view always shows up-to-date data! The database engine recreates the view, every time a user queries it.

CREATE VIEW Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2,...
FROM table_name
WHERE condition;
```

Crating a view from employee table

	emp_id	emp_name	emp_address	emp_contact	emp_salary	dept_id	desig_id	emptype_id	proj_id
1	1	Rahul	kathmandu	9845098976	50000	1	6	1	1
2	2	Gopal	bhaktapur	9845048976	60000	2	5	1	1
3	3	Kiran	dolakha	9845598976	70000	4	6	1	2
4	4	Prakriti	kathmandu	9845798976	40000	1	8	1	3
5	5	Samjhana	dang	9845898976	90000	5	7	2	1
6	6	Sudhir	surkhet	9845298976	30000	2	8	1	3
7	7	Sampanna	bhaktapur	9845398976	50000	1	6	2	2
8	8	Zakir	butwal	984578976	50000	3	6	2	1

Example 1: Create a view as Employee_Detail who are from out of Kathmandu valley

Query:

create view Employee_Detail as

select* from employee

where emp_address!='kathmandu' and emp_address!='bhaktapur';

Now we can treat this view as a separate table.

Example 2: Display all the data from view

Query: select* from Employee_Detail;

	emp_id	emp_name	emp_address	emp_contact	emp_salary	dept_id	desig_id	emptype_id	proj_id
1	3	Kiran	dolakha	9845598976	70000	4	6	1	2
2	5	Samjhana	dang	9845898976	90000	5	7	2	1
3	6	Sudhir	surkhet	9845298976	30000	2	8	1	3
4	8	Zakir	butwal	984578976	50000	3	6	2	1

Example 2: Display the name of employee according to salary in descending order

Query: `select emp_name as Employee_name, emp_salary as Salary
from Employee_Detail
order by emp_salary desc;`

	Employee_name	Salary
1	Samjhana	90000
2	Kiran	70000
3	Zakir	50000
4	Sudhir	30000

Example 3: Display the number of employees present in different job types

Query:

`select et.emptype_name as Job_Type,
COUNT(ed.emp_name) as NO_of_Employees
from Employee_Detail ed inner join employee_type et
on ed.emptype_id=et.emptype_id
group by et.emptype_name;`

	Job_Type	NO_of_Employees
1	Full Time	2
2	Part Time	2

Updating a View:

A view can be updated with the CREATE OR ALTER (REPLACE) VIEW statement.

CREATE OR ALTER VIEW Syntax

```
CREATE OR ALTER VIEW view_name AS
SELECT column1, column2,...
FROM table_name
WHERE condition;
```

Example: Update view Employee_Detail by adding the condition salary is greater than 50k

Query:

CREATE OR ALTER VIEW Employee_Detail as

select* from employee

where emp_address!='kathmandu' and emp_address!='bhaktapur' and emp_salary>50000;

Query: SELECT* FROM Employee_Detail;

Results		Messages							
	emp_id	emp_name	emp_address	emp_contact	emp_salary	dept_id	desig_id	emptype_id	proj_id
1	3	Kiran	dolakha	9845598976	70000	4	6	1	2
2	5	Samjhana	dang	9845898976	90000	5	7	2	1

Delete View:

A view is deleted with the DROP VIEW statement.

DROP VIEW Syntax

```
DROP VIEW view_name;
```

Query: Drop view Employee_Detail;

End of Unit-6